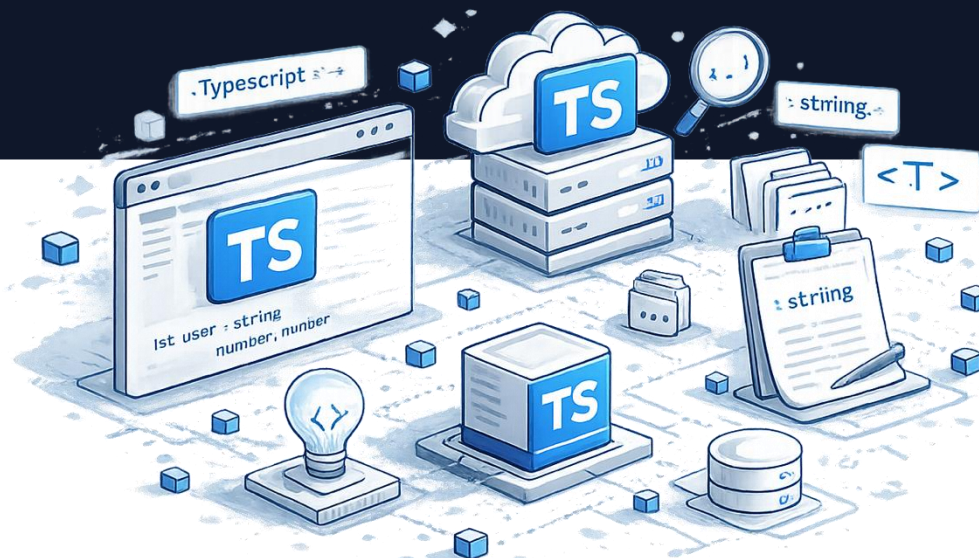




Progettazione di applicazioni web full stack TypeScript



Prof. Devis Bianchini
Dip. di Università degli Studi di Brescia



Argomento di questa lezione...

Cosa vedremo:

- Un breve «excursus» storico che ha portato a TypeScript
- Caratteristiche principali di TypeScript: tipizzazione, inferenza, classi, funzioni
- Tool e librerie esterne

A COSA ci serve:

- Tipi come contratto
- Errori intercettati prima
- Modellazione migliore di casi reali
- Riuso type-safe
- Codice più leggibile

Negli anni scorsi...



- Inizio anni 2000 – *Web «server-centrico»*
 - Gran parte della logica applicativa sul server
 - Il browser usato per rendering e piccole interazioni
 - JavaScript usato per validazioni semplici e piccoli effetti
- 2004-2006 – Diffusione di **AJAX** e **Rich Internet Applications**
 - Prodotti come webmail e mappe online rendono il modello popolare e alzano le aspettative degli utenti
- 2006-2010 – Librerie e strumenti per avere JavaScript su larga scala
 - Librerie come jQuery riducono il *caos cross-browser*
 - Tool di sviluppo del browser (ispezione DOM, console, debug)



- 2010+ - HTML 5 e *web «client-centrico»*
 - HTML5 e le nuove API del browser (storage, canvas, media, ecc.) spostano sempre più responsabilità sul client
 - App web più complesse, con logica, stato e UI dinamica
- **Standardizzazione** del linguaggio (ES5 → ES6/ES2015,...)
 - JavaScript evolve con nuove funzionalità (moduli, classi, arrow functions, ecc.)
 - Architetture più strutturate e grandi codebase, ma aumenta anche la complessità
- JavaScript *oltre il browser*
 - Runtime e toolchain (es. Node.js) rende JavaScript centrale anche nello sviluppo backend e negli strumenti di build



- Quando JS diventa il linguaggio per **applicazioni grandi e longeve**, emergono limiti tipici
 - **mancanza di tipi**: errori che si scoprono tardi (a runtime) e refactor rischiosi
 - **API e contratti poco espliciti** tra moduli/team
 - **scalabilità del codice**: mantenere coerenza e qualità diventa più difficile man mano che il progetto cresce

TypeScript nasce come «*estensione di Javascript*» per rispondere a queste esigenze

- aggiunge un *sistema di tipi* (per variabili e funzioni) e *strumenti di analisi* che aiutano a costruire e mantenere grandi codebase JavaScript in modo più sicuro e prevedibile
- È attualmente alla versione 5.9



- Negli anni si sono sviluppate Best Practice per la strutturazione del codice JavaScript che simulano funzioni mancanti
 - Prototype pattern, module pattern
- Script #
 - Permette la scrittura di codice in C# che viene compilato in JavaScript
 - Scarsa documentazione
 - Difficoltà ad utilizzare librerie non supportate
- Coffee Script
 - Necessità di imparare una nuova sintassi
 - Difficoltà ad utilizzare librerie non supportate



TypeScript

(definizione presa da <http://www.typescriptland.org/>)

- TypeScript is a language for application-scale JavaScript development

- TypeScript is a typed superset of JavaScript that compiles to plain JavaScript

- Any browser
- Any host
- Any OS
- Open Source

Tipizzazione



- Javascript è un linguaggio senza «*tipizzazione statica*» → errori rilevanti a *runtime*
- TypeScript permette la tipizzazione tramite opportuna sintassi
- *Intellisense* migliorato e errori a *compile time*
- Possibilità di utilizzare tipi opzionali usando ?

```
let a;           // tipo any  
let b: number;   // tipo number
```

Declare it

Annotate it

Set it

```
let num: number = 2;
```

Name it

Type it

Tipi di dichiarazione delle variabili



- **const**
 - non si può riassegnare la variabile (*costante*)
 - ha *block scope* (`{...}`)
 - ideale quando il riferimento non cambia
- **let**
 - si può riassegnare la variabile
 - ha *block scope*
 - utile per contatori, accumulatori, variabili che cambiano valore
- **var** (modalità «vecchia» pre-ES6)
 - ha *function scope*
 - la variabile è trattata come *undefined* fino alla riga di assegnazione (*hoisting* = «portare su»)

TypeScript: tipi primitivi e complessi



- **Primitivi**
 - *boolean*
 - *number* (tutti i tipi numerici)
 - *string* (delimitato da singolo o doppio apice)
 - *BigInt* (numeri più grandi di $2^{53}-1$, si usa il suffisso n a fine numero per istanziarlo)
- Tipi «speciali»
 - *any* (nessuna protezione)
 - *unknown* (protezione massima, richiede narrowing)
 - *never* (impossibile)
 - *undefined* (mancante, non assegnato)
 - *null* (assenza esplicita, scelta intenzionale)

Type inference



- «*Type inference*»: il compilatore riesce in molti casi a dedurre il tipo di variabile quando non viene dichiarato espressamente
- Possibile il casting tramite **as**
<nometipo>

```
// TypeScript infers 'string'
let username = "alice";

// TypeScript infers 'number'
let score = 100;

// TypeScript infers 'boolean[]'
let flags = [true, false, true];

// TypeScript infers return type as 'number'
function add(a: number, b: number) {
  return a + b;
}

// Cast a string
username as string;
```

TypeScript: array



```
const names: string[] = [];  
names.push("Dylan"); // no error  
// names.push(3); // Error: type 'number'  
  
const ro_names: readonly string[] = ["Dylan"];  
ro_names.push("Jack"); // Error: 'readonly string[]'.  
  
// Type inference  
const numbers = [1, 2, 3]; // inferred to type number[]  
numbers.push(4); // no error  
numbers.push("2"); // Error: Argument of type 'string'  
let head: number = numbers[0]; // no error
```

TypeScript: array



```
const names: string[] = [];  
names.push("Dylan"); // no error  
// names.push(3); // Error: type 'number'  
  
const ro_names: readonly string[] = ["Dylan"];  
ro_names.push("Jack"); // Error: 'readonly string[]'.  
  
// Type inference  
const numbers = [1, 2, 3]; // inferred to type number[]  
numbers.push(4); // no error  
numbers.push("2"); // Error: Argument of type 'string'  
let head: number = numbers[0]; // no error
```

La parola chiave **readonly** impedisce che l'array venga successivamente modificato

TypeScript: tuple



- ***Tupla*** = array di lunghezza pre-definita che può ospitare valori di tipo diverso

```
// define our tuple
let ourTuple: [number, boolean, string];

// initialize correctly
ourTuple = [5, false, 'Coding God was here'];
// We have no type safety in our tuple for indexes 3+
ourTuple.push('Something new and wrong');

// initialized incorrectly which throws an error
ourTuple = [false, 'Coding God was mistaken', 5];

// define our readonly tuple
const ourReadOnlyTuple: readonly [number, boolean, string]
= [5, true, 'The Real Coding God'];
// throws error as it is readonly.
ourReadOnlyTuple.push('Coding God took a day off');

const graph: [number, number] = [55.2, 41.3];
const [x, y] = graph;
```

TypeScript: tuple



- **Tupla** = array di lunghezza pre-definita che può ospitare valori di tipo diverso

A runtime funziona, ma TS continua a considerarla una tupla `[number, boolean, string]`, c'è un elemento extra che il tipo non descrive

```
// define our tuple
let ourTuple: [number, boolean, string];

// initialize correctly
ourTuple = [5, false, 'Coding God was here'];
// We have no type safety in our tuple for indexes 3+
ourTuple.push('Something new and wrong');

// initialized incorrectly which throws an error
ourTuple = [false, 'Coding God was mistaken', 5];

// define our readonly tuple
const ourReadonlyTuple: readonly [number, boolean, string]
= [5, true, 'The Real Coding God'];
// throws error as it is readonly.
ourReadonlyTuple.push('Coding God took a day off');

const graph: [number, number] = [55.2, 41.3];
const [x, y] = graph;
```

TypeScript: tuple



- **Tupla** = array di lunghezza pre-definita che può ospitare valori di tipo diverso

L'ordine è importante

```
// define our tuple
let ourTuple: [number, boolean, string];

// initialize correctly
ourTuple = [5, false, 'Coding God was here'];
// We have no type safety in our tuple for indexes 3+
ourTuple.push('Something new and wrong');

// initialized incorrectly which throws an error
ourTuple = [false, 'Coding God was mistaken', 5];

// define our readonly tuple
const ourReadonlyTuple: readonly [number, boolean, string]
= [5, true, 'The Real Coding God'];
// throws error as it is readonly.
ourReadonlyTuple.push('Coding God took a day off');

const graph: [number, number] = [55.2, 41.3];
const [x, y] = graph;
```

TypeScript: object



- **Object** =
elemento di
tipo non
primitivo che
contiene
proprietà (a
loro volta
dotate di tipo)

```
const car: { type: string, model: string, year: number } = {  
  type: "Toyota",  
  model: "Corolla",  
  year: 2009  
};  
  
const new_car = {  
  type: "Toyota",  
};  
new_car.type = "Ford"; // no error  
new_car.type = 2; // Error: Type 'number' is not assignable to type 'string'.  
  
const my_car: { type: string, mileage: number } = {  
  // Error: Property 'mileage' is missing in type '{ type: string; }'  
  type: "Toyota",  
};  
  
const right_car: { type: string, mileage?: number } = { // no error  
  type: "Toyota"  
};  
  
const nameAgeMap: { [index: string]: number } = {};  
nameAgeMap.Jack = 25; // no error  
nameAgeMap.Mark = "Fifty"; // Error: Type 'string' is wrong.
```

TypeScript: object



- **Object** = elemento di tipo non primitivo che contiene proprietà (a loro volta dotate di '')

Type inference funziona anche con gli objects

```
const car: { type: string, model: string, year: number } = {
  type: "Toyota",
  model: "Corolla",
  year: 2009
};

const new_car = {
  type: "Toyota",
};
new_car.type = "Ford"; // no error
new_car.type = 2; // Error: Type 'number' is not assignable to type 'string'.

const my_car: { type: string, mileage: number } = {
  // Error: Property 'mileage' is missing in type '{ type: string; }'
  type: "Toyota",
};

const right_car: { type: string, mileage?: number } = { // no error
  type: "Toyota"
};

const nameAgeMap: { [index: string]: number } = {};
nameAgeMap.Jack = 25; // no error
nameAgeMap.Mark = "Fifty"; // Error: Type 'string' is wrong.
```

TypeScript: object



- **Object** = elemento di tipo non primitivo che contiene proprietà (a loro volta dotate di tipo)

C'è un solo modo corretto che permette l'inserimento di proprietà opzionali

```
const car: { type: string, model: string, year: number } = {
  type: "Toyota",
  model: "Corolla",
  year: 2009
};

const new_car = {
  type: "Toyota",
};
new_car.type = "Ford"; // no error
new_car.type = 2; // Error: Type 'number' is not assignable to type 'string'.

const my_car: { type: string, mileage: number } = {
  // Error: Property 'mileage' is missing in type '{ type: string; }'
  type: "Toyota",
};

const right_car: { type: string, mileage?: number } = { // no error
  type: "Toyota"
};

const nameAgeMap: { [index: string]: number } = {};
nameAgeMap.Jack = 25; // no error
nameAgeMap.Mark = "Fifty"; // Error: Type 'string' is wrong.
```

TypeScript: enum



- **Enum** =
rappresenta
un gruppo o
insieme di
costanti
(valori non
modificabili)

```
enum CardinalDirections {  
    North = 1,  
    East,  
    South,  
    West  
}  
  
// logs 1  
console.log(CardinalDirections.North);  
// logs 4  
console.log(CardinalDirections.West);  
  
enum StringCardinalDirections {  
    North = 'North',  
    East = "East",  
    South = "South",  
    West = "West"  
};  
  
// logs "North"  
console.log(StringCardinalDirections.North);  
// logs "West"  
console.log(StringCardinalDirections.West);
```

TypeScript: enum



- **Enum** =
rappresenta
un gruppo o
insieme di
costanti
(valori non
modificabili)

Esistono solo **enum** per numeri
e stringhe

Nel caso numerico, è previsto
un meccanismo di **auto-
increment** (sempre di 1)

```
enum CardinalDirections {  
    North = 1,  
    East,  
    South,  
    West  
}  
  
// logs 1  
console.log(CardinalDirections.North);  
// logs 4  
console.log(CardinalDirections.West);  
  
enum StringCardinalDirections {  
    North = 'North',  
    East = "East",  
    South = "South",  
    West = "West"  
};  
  
// logs "North"  
console.log(StringCardinalDirections.North);  
// logs "West"  
console.log(StringCardinalDirections.West);
```



- È possibile creare **classi** con campi, proprietà, costruttori e funzioni
- Valgono i comuni modificatori di visibilità (**public**, **private**, **protected**)
- La parola chiave **readonly** può essere usata per le proprietà di una classe

```
class Car {  
    // Property (public by default)  
    engine: string;  
  
    // Constructor  
    // (accepts a value to  
    // initialize the engine)  
    constructor(engine: string) {  
        this.engine = engine;  
    }  
}  
  
var hondaAccord = new Car("V6");
```



- TypeScript semplifica l'«ereditarietà» usando **extends**, che permette l'accesso ai membri pubblici e al costruttore
- Il costruttore può essere ridefinito
- Per chiamare il costruttore della classe base si usa **super**

```
class Truck extends Car {  
  
    bigTires: boolean;  
    constructor (engine:string,  
        bigTires:boolean) {  
        super(engine);  
        this.bigTires = bigTires;  
    }  
}  
  
var truckAccord = new Truck("V6",true);
```



- TypeScript permette di definire tipi complessi sotto forma di *interfacce*

```
interface ICar {  
    engine: string;  
    color: string;  
}  
  
class Car implements ICar {  
    engine: string;  
    color: string;  
  
    constructor (engine: string, color:string) {  
        this.engine = engine;  
        this.color = color;  
    }  
}  
  
var auto = new Car("V6","red");
```

Functions e arrow functions



- **Arrow functions** → è un modo alternativo per definire le funzioni

```
var myFunc1 = function (h:number,  
    | w:number) {  
    return h*w;  
};  
  
// può essere scritto come  
var myFunc2 = (h:number, w:number) => h*w;  
  
myFunc1(2,3);  
myFunc2(2,3);
```

Le seguenti espressioni sono equivalenti:

```
(x) => {return Math.sin(x);}  
(x) => Math.sin(x);  
x => {return Math.sin(x);}  
x => Math.sin(x);
```

Functions e arrow functions



- Le «*arrow functions*» risolvono il problema dello scope del **this**
 - Con una arrow function, **this** non cambia, resta quello del punto in cui la funzione è stata scritta
 - Con **function**, invece, **this** dipende da chi la chiama (e nelle callback spesso «si perde»)

```
const obj = {  
  name: "Alice",  
  sayLater() {  
    setTimeout(() => console.log(this.name), 100); // ✓ "Alice"  
    setTimeout(function () { console.log(this.name); }, 100); // ✗ this "perso"  
  }  
};
```



- I moduli in TypeScript permettono una migliore scrittura del codice, favorendo il riuso
- Un modulo è un file (o pacchetto) che:
 - *esporta* ciò che vuole rendere pubblico
 - *importa* ciò che gli serve da altri moduli
 - crea un proprio *scope*, senza «inquinare» quello *globale*



Progettazione di applicazioni web full stack TypeScript



Prof. Devis Bianchini
Dip. di Università degli Studi di Brescia